A job ready bootcamp in C++, DSA and IOT

Operators in C Language



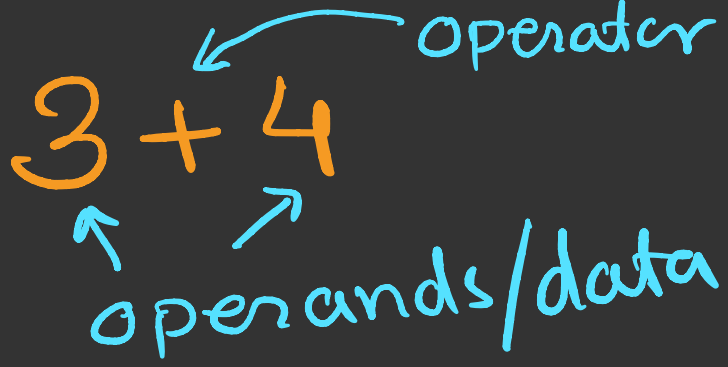
Saurabh Shukla (MySirG)

Agenda

1. Arithmetic Instruction
2. Classification of operators
3. Unary operators
4. Arithmetic operators
5. Bitwise operators
6. Relational operators
7. Logical operators
8. Assignment operators

Arithmetic Instruction

An instruction which is used to manipulate data using operators, is known as Arithmetic Instruction.



1. Unary
2. Binary
3. Ternary

3 + 4 * 5

= 23 35

Classification of Operators

1. Unary operators — +, −, ++, −−, sizeof()
2. Arithmetic operators — *, /, %, +, −
3. Bitwise Operators — &, |, ^, ~, >>, <<
4. Relational Operators — <, >, <=, >=, ==, !=
5. Logical Operators — !, &&, ||
6. Conditional Operator — ?:
7. Assignment Operators — =, +=, -=, *=, /=, %=

Unary Operaters

+, -, ++, --

+5

-7

x--; post decrement

--x; pre decrement

x = x-1

Increment operator

++

3++; error

```
int x=5;
printf("%d",x);
x++;
printf("%d",x);
++x;
printf("%d",x);
```

x: 7 ~~5~~ ~~6~~

5

x++; ← x = x+1 post increment

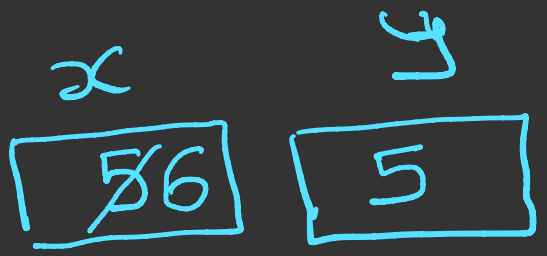
6

++x; ← x = x+1 pre increment

7

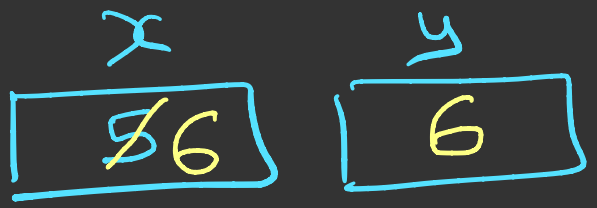
Find Output of the following program?

```
#include<stdio.h>
int main()
{
    int x=5, y;
    y=x++;
    printf("%d %d", x, y);
}
```



6 5

Find Output of the following program?




```
#include<stdio.h>
int main()
{
    int x=5, y;
    y=++x;
    printf("%d %d", x, y);
}
```

6 6

Unary Operators

sizeof()

1. Data type
2. Variable
3. Constant

- Integer constant is of type int
- Real constant is of type double
- Character constant is of type int

Type modifiers

short

long

```
short int x;
short x;
```

char	1 byte
short	2 bytes
int	4 bytes
long	8 bytes

Arithmetic Operators

Associativity Rule : L to R

* / %

\+ -

3+4 7

3-4 -1

3*4 12

3/4 0

11/4 2

15%6 3

21%7 0

426%10 6

3%4 3

a * b / c *

a / b * c /

a + b * c *

3/4 → 0.75 ← maths

Integer / Integer → integer

```
4) 3 (0
   0
   -
   3
```

3.5%2 Error

3.0 / 4 → 0.75

3 / 4.0 → 0.75

3.0 / 4.0 → 0.75

Bitwise Operators

& , | , ^ , ~ , >> , <<

```
int x;
x = 35 & 52;
```

0 & 0 → 0	0 ^ 0 → 0
0 & 1 → 0	0 ^ 1 → 1
1 & 0 → 0	1 ^ 0 → 1
1 & 1 → 1	1 ^ 1 → 0
0 | 0 → 0	~1 → 0
0 | 1 → 1	~0 → 1
1 | 0 → 1	
1 | 1 → 1	

35 = 00100011

52 = 00110100

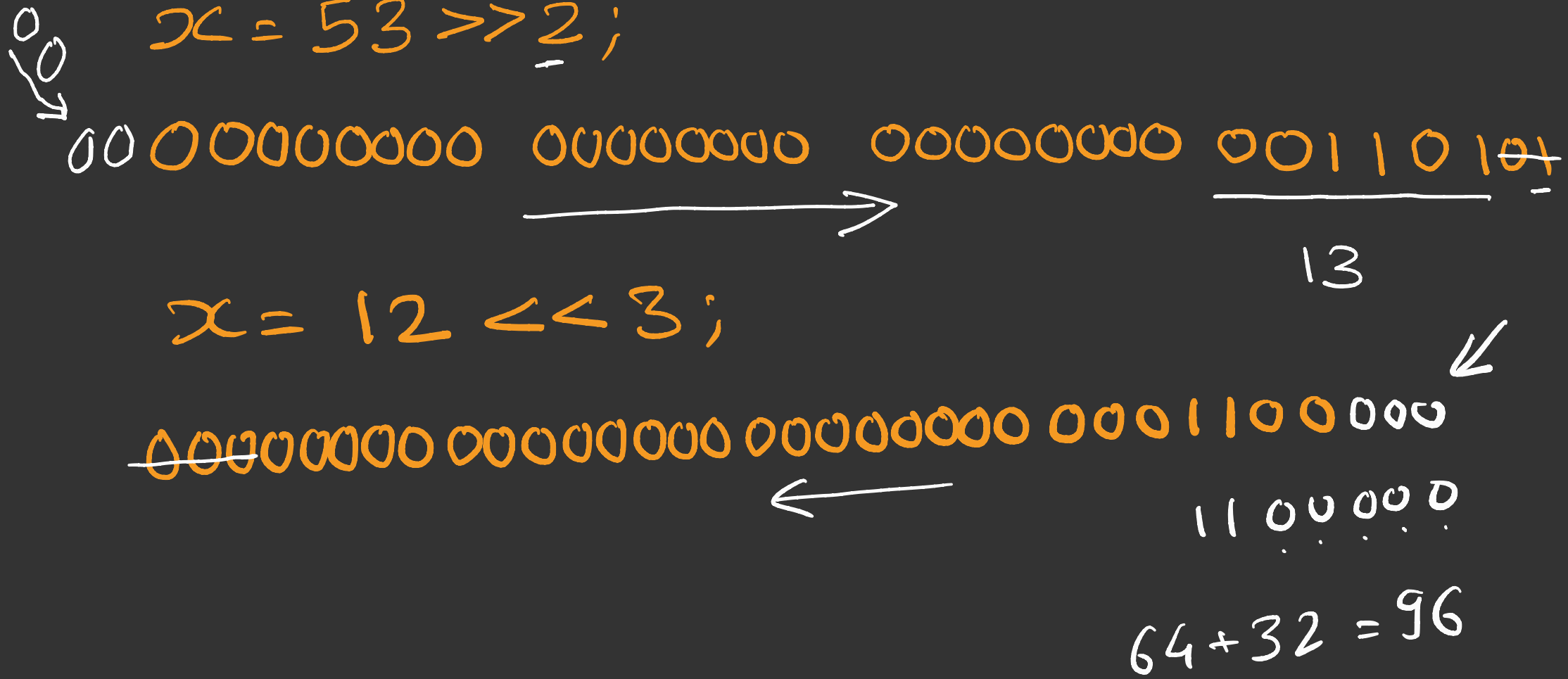
32 = 00100000

```
x = 44 | 68;
```

44 = 00101100

68 = 01000100

108 = 01101100

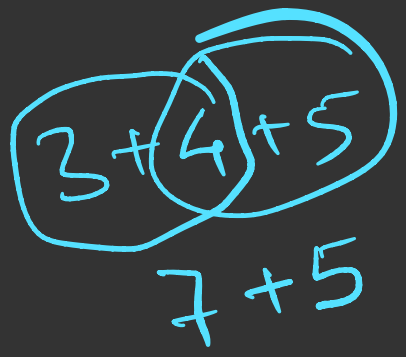

x = 53 >> 2;
00000000000000000000000000110101
13
x = 12 << 3;
00000000000000000000110000000
1100000
64+32 = 96

Relational Operators (L→R)

<, >, <=, >=,
==, !=

Always give result as 0 or 1

True → 1
False → 0



0 x = 3 > 4;

1 x = 5 <= 5;

0 x = 4 != 4;

0 x = 10 > 8 > 6;

1 > 6

1 x = 3 != 4;

x = 2 == '2'

0 x = 2 == 50

Logical Operators

!	NOT (unary)
&&	AND
||	OR

Every non-zero value is True

Zero is False

! True → False (0)

! False → True (1)

exp1 && exp2	
T && T	→ T
T && F	→ F
F && X	→ F

exp1 || exp2	
F || F	→ F
F || T	→ T
T || X	→ T

```
x = ! 5 > -2 ;
    ! 5
    ! 0 > -2
    ! (5 > -2)
    ! 1
    0
```

$x > 0$	&&	$y > 0$	
T		T	→ T
T		F	→ F
F		T	→ F
F		F	→ F

Assignment Operators

R to L

=

$x = 4$; ✓

$4 = x$; ✗

$x = y = 4$;

Variable =

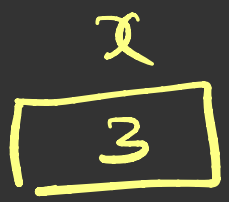
int x = 3;

x = x + 5;

Container

Content

= 3 + 5



Compound Assignment Operators

+=, -=, *=, /=, %=

```
int x = 5;
x += 3;  → x = x + 3
x -= 4;  → x = x - 4;
x *= 7;  → x = x * 7
x /= 5;  → x = x / 5
x %= 2;  → x = x % 2
```